

Analysis: Alphabet Cake

Alphabet Cake: Analysis

A brute force cell filling Small approach

In the Small dataset, the cake can consist of at most 12 units. A viable brute-force strategy is to try all ways of filling in every blank cell with each letter already appearing on the cake. If L letters are already present, then we have L possibilities for each of the remaining $12-L$ cells. Then, the number of combinations to try is $L^{(12-L)}$, which is a lot less than 12^{12} . In this case, since L can range from 1 to 12, inclusive, the maximum value that this can take is $5^7 = 78125$, which is a pretty small number for a computer.

One way to check a cake is to first make one pass through the cake, going left-to-right, top-to-bottom, and find each letter's upper leftmost position, lower rightmost position, and frequency. Then, for each letter, check that the rectangle defined by the upper leftmost position and lower rightmost position contains only that letter, and that its area equals the frequency count.

A brute force rectangle-extending Small approach

Another possible strategy is to start at a letter, draw a bounding box around it that includes only ?s, move on to another letter, and so on. In the Small dataset, there are relatively few choices for how to draw these bounding boxes. As long as you use other pre-existing letters to prune the possible bounding boxes for each letter, and you carefully check for overlaps and unused cells, an exhaustive search should be fast enough.

There are non-exhaustive strategies similar to this that have the potential to fail, though. Consider this algorithm: for each letter, start with a 1-by-1 bounding box containing only that letter. Stretch the box as far as it will go to the top and bottom without hitting other letters, then stretch the box as far as it will go to the left and right without hitting other letters. Depending on the order in which you handle the letters, this can fail. For example, consider this case:

```
A?B
C??
??D
?EF
```

If the algorithm proceeds in left-to-right, top-to-bottom order, it will fill up the grid as follows, leaving an unfilled ?:

```
AAB
C?B
CDD
CEF
```

A recursive Large approach

Fittingly enough for a problem about cake, there is a simple divide-and-conquer strategy for the Large dataset. If the cake has only one letter on it, we can fill it up with that letter. Otherwise, we can make a horizontal or vertical cut that divides the cake into two sub-cakes, such that each part has at least one letter on it; this creates two more instances of the same problem. It is always possible to do this if there are at least two letters on the cake; one surefire way is to make a dividing cut that includes the right border of the leftmost of a pair of letters, or, if they are in the same column, the bottom border of the topmost of the pair.

A greedy Large approach

There is a simple non-recursive approach as well. First, within each row, we can extend each existing letter into all cells to the right of that letter, until we reach another existing letter or the edge of the cake. Then, we can extend the leftmost existing letter (if any) into all cells to the left of that letter.

At this point, any rows that had no existing letters are still empty. We can make a pass through the grid from the second row to the bottom row, replacing each empty row with the row above it. Then we can make one more pass from the next-to-last row up to the top row, replacing each empty row with the row below it.

It is not difficult to prove that it is impossible for this strategy to produce a non-rectangular region for a letter, to produce disjoint regions for a letter, or to leave any cell unfilled.

Analysis: Play the Dragon

Play the Dragon: Analysis

Small dataset

The Small limits are large enough to foil pure simulation of all possible choices, so we need to have some insights before proceeding.

- The dragon should cure only when it is forced to — that is, when the knight's next attack would defeat it, and attacking or debuffing would not prevent that. Otherwise, it is always better to do something else.
- All buffs should come before all attacks, so that each of the dragon's attacks gets the benefit of all of the buffs.
- The number of buffs directly determines the number of attacks needed.
- All debuffs should come before all buffs/attacks, so that the total amount of damage the dragon must withstand is minimized.
- If the knight's first attack will defeat the dragon even if the dragon attacks or debuffs in the first turn, the case is impossible.
- If the dragon is forced to cure two turns in a row, then the case is impossible, since that implies that the dragon will have to cure every turn.

These observations add up to a strategy: spend some number D' of turns debuffing, then some number B' of turns buffing, then some number A' of turns attacking, and interleave cures only as needed to not be defeated. Since B' determines A' , we only need to consider (D', B') pairs. Since A_k cannot exceed 100, there is no reason to ever do more than 100 debuffs or 100 buffs; moreover, the worst-case scenario can't possibly require more than a couple hundred turns ($D' + B' + A'$). We can place much smaller upper bounds than those with a little more thought, but it is already clear that direct simulation should be fast enough for the Small dataset.

So, we can proceed with translating the above strategy into code. We must take care to prioritize actions in the right order. In particular, we must avoid curing when we do not need to or failing to cure when we should. Once that is written, we can simulate each possible (D', B') pair and find the global minimum number of turns, or determine that the case is IMPOSSIBLE.

Large dataset

We noted above that all debuffs should come before all buffs/attacks, and that the number of buffs determines the number of attacks. In fact, the buff/attack part of the problem is independent of the debuff part of the problem. Changing the number of debuffs may change the number of cures, but regardless of how many times we debuff, we have nothing to gain by using more than the minimum number of buff + attack turns; that would just make us waste more turns curing.

We can find this minimum number of buff + attack turns as follows. First, we suppose that we will buff 0 times, and we determine the total number of attacks needed to defeat the knight. Then, we can repeatedly increase B' by 1 and calculate the number of attack turns A' required at that new level of attack power. As soon as this causes the total to get *larger*, we can stop (and take the previous total). It is safe to stop at that point because the total number of turns is given by

$$B' + \text{ceil}(H_k / (A_d + B' \tilde{A} - B))$$

The B' part contributes a line with positive slope; the rest contributes a decaying step function. If that step function were a smooth curve, there would be one point at which the rate of decrease from the curve exactly matched the rate of increase from the linear part, and the function would take on our desired minimum there. Because of the discretization, there may actually be multiple values of B' that yield the minimum number of $B' + A'$ turns, but it does not matter which one we choose; only the total matters.

Finding the minimum $B' + A'$ in this way takes $O(\sqrt{N})$ time, where N is the common upper limit for all of the parameters (10^9 for the Large). This is because once we have raised B' to about $\sqrt{N}$, we can defeat the knight in about $\sqrt{N}$ attack rounds, and there is no need to buff further. It is also possible to solve this part of the problem using binary search or ternary search, or by solving a quadratic equation.

What about the number D' of debuffs? The key observation here is that we do not need to consider every possible value of D' . For instance, suppose that H_d is 100, $A_k = 50$, and $D = 1$. Reducing A_k to 49 (which takes 1 turn of debuffing) is as good as reducing A_k to 48 or 34; in all of these cases, the dragon has to cure every other turn. However, reducing A_k to 33 (which takes 17 turns of debuffing) means that the dragon only has to cure on every third turn. We might as well only consider these threshold values of $D' = 0, 1, 17, 26, 31, \dots$; we can leave out the others. We can find each of these values formulaically in constant time.

Once we have these values, we do not even need to simulate them independently. Note that a simulation with $D' = 17$ begins by looking like a simulation with $D' = 1$, since we have to do one debuff before we do the other sixteen. So we can perform a single simulation in which we keep track of a number T of turns used so far, and repeat the following:

- Pretend that we will do no more debuffing. Figure out how many additional turns are needed to buff + attack while curing enough to survive. Compare the total (T plus that number) to the best total we have seen so far.
- Figure out how many turns are needed to increase the number of debuffs to the next threshold value, while curing enough to survive. Add that number to T .

It takes additional turns to debuff more, but that debuffing may "pay for itself" by saving curing turns during the buff + attack phase. Our strategy will find the right balance.

Instead of actually simulating the turns, we can take advantage of the way we have chosen threshold values of D' : in the debuffing period between two of our values of D' , the frequency of curing remains constant. So we can calculate the total number of debuff turns + cures via a formula, and we can do the same for the number of cures in the buff + attack phase. With this optimization, the complexity of this step is $O(\sqrt{N})$; since we also took $O(\sqrt{N})$ time to find the optimal number of $B' + A'$ turns, the algorithm is $O(\sqrt{N})$ overall.

All that remains is to actually implement the above, which is perhaps even harder than coming up with the algorithm; there are many opportunities to make off-by-one errors! Although this is not generally the case in Code Jam, in this particular problem, the limits do allow solutions with some but not all of the above insights, and additional low-level optimizations, to pass within the 8-minute window.

Analysis: Ratatouille

Ratatouille: Analysis

Small dataset

The number of grams in a given package is not important in and of itself; what is important is which multiples of the recipe that package could be used in. If the base recipe requires 10 g of tomato, for instance, then a 201 g package of tomatoes can be used to make 19, 20, 21, or 22 times the recipe, but not any other multiple. So we can think of this package as the inclusive range [19, 22].

How can we find these ranges? Let Q be the amount in the package and R be the amount of that ingredient required by the base recipe. Then any allowable multiple m must satisfy both $0.9 \times R \times m < Q$ and $Q < 1.1 \times R \times m$. Simplifying and rearranging this, we get $(10/11) \times (Q/R) < m < (10/9) \times (Q/R)$; any integer value of m that satisfies this inequality is part of the range, and we can find the lower and upper bounds of this range in constant time. (Note that a range might not include *any* integer values, e.g., for $R = 10$ and $Q = 15$.) It is possible to use integer division here and avoid floating point numbers entirely.

In the Small dataset, there can be at most two ingredients. If there is one ingredient, the problem is very simple: each package with a non-empty range (as described above) forms a kit, and each package with an empty range does not.

What if there are two ingredients? Then we need to pair up the packages into kits, but we must be careful, since not any assignment will be optimal. For example, suppose our recipe requires 5 g of A and 10 g of B, and we have the following packages: 45 and 50 of A, and 110 and 111 of B. Converting these to ranges, we get [9, 10] and [10, 11] for A, and [10, 12] and [11, 12] for B. If we pair the [10, 11] package of A with the [10, 12] package of B, then we will be left with a [9, 10] package of A and an [11, 12] package of B, and those cannot be paired. However, we can make two kits by pairing the [9, 10] package of A with the [10, 12] package of B, and the [10, 11] package of A with the [11, 12] package of B.

We could solve the problem via bipartite matching, but it is not needed. Since there can be at most 8 packages of each ingredient in the Small, it is possible to use brute force to try all 8! possible matchings and find the largest number of kits that can be formed.

Large dataset

The following greedy strategy works for the Large, for any number of ingredients:

- Keep making the smallest possible multiple of the recipe that we can.
- Whenever we have a choice of packets, we choose one with the smallest upper end of the range. Since we are always making the smallest multiple we can, we do not care about the parts of the ranges up to and including that multiple. We only care about how large the upper ends get. Since the ranges are continuous, a range with a larger upper end is strictly more flexible for future pairings than a range with a smaller upper end.

This strategy would not necessarily be optimal for arbitrary ranges, but it is in this case thanks to the following property: if I_1 and I_2 are the ranges of valid multipliers for the same ingredient, and I_1 is included in I_2 , then I_1 and I_2 have at least one endpoint in common. This is not hard to

prove: if I_i comes from an original packet of size S_i , if $S_1 \leq S_2$, then the lower bound of I_1 must be no greater than the lower bound of I_2 . Otherwise, $S_1 > S_2$, so the upper bound of I_1 must be no less than the upper bound of I_2 . Let us call this property 0.

We need some preliminary properties of our greedy algorithm before the main proof:

- *Property 1:* Whenever it chooses to discard a range, then there was no valid kit remaining to use that range for. This follows directly by the condition to discard ranges.
- *Property 2:* Whenever it chooses to use a set of ranges to form a kit, and M is the lower bound of the intersection of all those ranges, there is no other set of ranges left that could possibly make a kit with a multiplier strictly less than M . This follows directly from the order in which the ranges are considered.

Now we can combine all those properties and proceed by contradiction. Let $M_1, M_2, \dots, M_k$ be the smallest multipliers of the kits produced by our greedy algorithm, in non-decreasing order. Let $N_1, N_2, \dots, N_k, N_{k+1}$ be the smallest multipliers of an assignment that produces one more kit, in non-decreasing order.

- Case A: Let i be the minimum i for which $N_i < M_i$. By property 2, that can only happen if there is no set of ranges to make a kit with multiplier N_i . That means the greedy algorithm discarded such ranges or used them for some previous kit. Property 1 prevents the discarding. Property 0 and the processing order prevented the algorithm from using it before, because if a range R is chosen at some point, all other remaining ranges for that ingredient have an upper bound that is no less than R 's, and therefore, such choice can't prevent a future kit for being formed; all other ranges are, at that point, at least as useful as R for larger multipliers.
- Case B: There is no such i . Again, the greedy algorithm must have some range missing not to be able to produce the kit with multiplier N_{k+1} . However, the same argument as in Case A using properties 1 and 0 shows that the wrong earlier decision that made that range not available couldn't have happened.

We can implement this strategy with a separate list for each ingredient, sorted first by nondecreasing lower end of range and then by nondecreasing upper end of range. We keep looking at the set of earliest ranges of the lists. If they all have an intersection, make them into a kit and remove them. Otherwise, remove the least useful range (the one with the smallest upper limit). We can speed this process up by using a priority queue, but with $N \times P \leq 1000$, such optimizations are not necessary.